A Tutorial on SQL Window Functions being DuckDB.

12 March 2024 · data engineering

Working with SQL Window Functions

🤓 TL;DR: Window functions are among the most powerful and useful features of any SQL query engine. However, the declarative nature of SQL can make them feel counterintuitive when you first start working with them. In this guide, I will demonstrate the beauty of SQL windows and show that they are actually much less intimidating than you might think (and even fun!).

SQL Window FunctionsSQL Window Functions

Window functions in SQL allow analytical functions to execute on a specified window of rows relative to the current row. Unlike aggregations, Windowfuncs, on the other hand, offers more than 70 functions that can be used with SQL windows. Postgresql also supports 11 SQL aggregations that can be used as window functions. Plus, custom functions that are packaged by default, in addition to any user-provided aggregation function.

Hopefully, the above introduction has captured your attention and helped you realize how important SQL windows are, based on the efficient database vendors are willing to add support for them.

But what's a window in SQL?

The concept of windows is actually pretty simple. It allows us to perform calculations across sets of rows that are related to the current row in some way.

Think of (learning through a) the cone but the calculation we want to perform is related to the last 2 (and to the current one makes 3) rows in a group of rows ordered by time.

Another way to think about window functions is by considering the `OVER()` operator. Here on our `OVER()` we can write SQL to compute a function by grouping first the data using the parameters of the `OVER()` clause. Consider the following SQL:

```
select sum(m.c) over(partition by t.name order by t.ts asc) from t1 t
inner join t2 m on (t.id = m.id);
```

In the above example, we ask SQL to split events among unique name,ts and sum them for each user-specified window. Both the calculation and also the grouping results will be included in the resulting table. So for example:

name_id	event
1	click
1	click
2	load
1	load
2	load

Assuming the above input table, the result of the query will look like this:

name_id	total_actions
1	3
2	2

You can think of SQL Windows as something similar to how data is processed but without having necessarily to prevent the results grouped at the end.

But windows are actually even more powerful as we will see.

What comes after partitioning ?

As we've seen previously, partitioning is a first important concept to understand about windows. We can create subsets from our data based on the partitioning clause and then execute functions the main operation defining the window. Now, it's time for us to learn how to partition the data.

But we can do more than that! See the example below.

Here's an example SQL query that uses a window function and the `lag()` function to calculate the time lapse between consecutive events:

```
select
    event_time,
    event_type,
    product_id,
    lag(event_time, -1)over(partition by user_id order by event_time) as [event_time_lead]
from
    [dbo].[events]
```

This query partitions the data by `user_id` and orders it by `event_time`. Remember what we said previously about partitioning? Well, in this case, we are using the partitioning to create the partitioned the work and we have to wo will partition.

We will also sort our data based on `event_time`. The reason we do so is because we want to calculate the time lapse between consecutive events. The reason we are sorting is because of what we'll do next.

But query then uses the `lag()` function, which is the window function that will do the magic for us. What `lag()` does, is to make available the previous value to the current row as currently process, within our window.

The code part:

```
lag(event_time,-1)over(...)
```

Does exactly that, while we go through the current row's `event_time`, we get access to the `event_time` value from the previous row. Because we have ordered our data, we can be confident that the values and calculate the time lapse!

The window magic is defined in this part of the syntax:

```
lag(event_time, -1)over(partition by user_id order by event_time)
```

The `lag()` function indicates the start of the window definition, just try to read this as a sentence written in English and it should explain what is happening. That's part of the beauty of being a declarative language!

The difference is going to be calculated based on the `event_time` column, and we want to do this for each unique `user_id`. We have used and we also sort this and based on the event_time. What is important to remember here is that sorting is not global, instead it will be sorted only according each individual set defined by the `user_id`.

After the partitions have been created and sorted, the query engine starts iterating the rows of each partition and we can use the `LAG` function will make the value of the previous one available to the engine.

At this point, everything is available for the engine to execute the function and calculate the new values and that's exactly what it does!

`lag()` and the above example is a great introduction to the last important concept about windows in SQL. Framing!

Windows and Frames

Windowing brings us a relation into independent groups or "partitions," that enables these partitions and computes a new column for each row based on nearby values.

In many cases, the functions that we apply depend only on the current row's partition. In other cases, we want to be able to remain fixed anchors as we move through.

In other cases though, the function might need access to some of the previous or following values. This was the case with `LAG` in our previous example.

Although we had defined a partition based on the `user_id`, we also needed to provide to our function for this case subtraction with the previous to the current value. This is exactly what `LAG` does.

Frames are a generalization of this concept.

A frame is a number of rows or either before (preceding or following) of the current row.

In our previous example, the frame was one row preceding our current row. Although we didn't provide how many preceding rows we wanted, the default value is 1 and the default which is 0. But we can use any number as well.

In DuckDB the definition of `LAG` is:

```
lag(arg0 any, [offset bigint, [default any ]])
```

Offset refers to the number of rows preceding the current one. Default is the value that will be used if the offset goes to a row outside of the requested offset does not exist. For example, if we are at the first one and want to access the previous one, we can define a default value to return instead.

Let's consider the following table:

```
select * from t1;
```

name_id	user_id	t_timestamp
1	1	"2023-01-01 00:00:00"+0000"
2	1	"2023-01-01 00:00:01"+0000"
3	1	"2023-01-01 00:00:02"+0000"
4	1	"2023-01-01 00:00:03"+0000"
5	1	"2023-01-01 00:00:04"+0000"
6	1	"2023-01-01 00:00:05"+0000"
7	1	"2023-01-01 00:00:06"+0000"
8	1	"2023-01-01 00:00:07"+0000"
9	1	"2023-01-01 00:00:08"+0000"
10	1	"2023-01-01 00:00:09"+0000"
11	1	"2023-01-01 00:00:10"+0000"
12	1	"2023-01-01 00:00:11"+0000"
13	1	"2023-01-01 00:00:12"+0000"
14	1	"2023-01-01 00:00:13"+0000"
15	1	"2023-01-01 00:00:14"+0000"
16	1	"2023-01-01 00:00:15"+0000"
17	1	"2023-01-01 00:00:16"+0000"
18	1	"2023-01-01 00:00:17"+0000"
19	1	"2023-01-01 00:00:18"+0000"
20	1	"2023-01-01 00:00:19"+0000"

Below celebrated the concept of partitions using frames, let's see how the table will look like if we apply a window like the following:

```
select*from t1 t,lag t.timestamp over partition by user_id order by t.timestamp);
```

In the table below you can see how it will look like after the application of `PARTITION BY`:

```
select*from t1 t,lag t.timestamp over partition by user_id order by t.timestamp);
```

name_id	user_id	t_timestamp	lag(t.timestamp)
1	1	"2023-01-01 00:00:00"+0000"	NULL
2	1	"2023-01-01 00:00:01"+0000"	"2023-01-01 00:00:00"+0000"
3	1	"2023-01-01 00:00:02"+0000"	"2023-01-01 00:00:01"+0000"
4	1	"2023-01-01 00:00:03"+0000"	"2023-01-01 00:00:02"+0000"
5	1	"2023-01-01 00:00:04"+0000"	"2023-01-01 00:00:03"+0000"
6	1	"2023-01-01 00:00:05"+0000"	"2023-01-01 00:00:04"+0000"
7	1	"2023-01-01 00:00:06"+0000"	"2023-01-01 00:00:05"+0000"
8	1	"2023-01-01 00:00:07"+0000"	"2023-01-01 00:00:06"+0000"
9	1	"2023-01-01 00:00:08"+0000"	"2023-01-01 00:00:07"+0000"
10	1	"2023-01-01 00:00:09"+0000"	"2023-01-01 00:00:08"+0000"
11	1	"2023-01-01 00:00:10"+0000"	"2023-01-01 00:00:09"+0000"
12	1	"2023-01-01 00:00:11"+0000"	"2023-01-01 00:00:10"+0000"
13	1	"2023-01-01 00:00:12"+0000"	"2023-01-01 00:00:11"+0000"
14	1	"2023-01-01 00:00:13"+0000"	"2023-01-01 00:00:12"+0000"
15	1	"2023-01-01 00:00:14"+0000"	"2023-01-01 00:00:13"+0000"
16	1	"2023-01-01 00:00:15"+0000"	"2023-01-01 00:00:14"+0000"
17	1	"2023-01-01 00:00:16"+0000"	"2023-01-01 00:00:15"+0000"
18	1	"2023-01-01 00:00:17"+0000"	"2023-01-01 00:00:16"+0000"
19	1	"2023-01-01 00:00:18"+0000"	"2023-01-01 00:00:17"+0000"
20	1	"2023-01-01 00:00:19"+0000"	"2023-01-01 00:00:18"+0000"

And this is how the table looks like after we order it. If you notice you will see that partitioning exists only inside each partition and it's not global.

name_id	user_id	t_timestamp	lag(t.timestamp)
1	1	"2023-01-01 00:00:00"+0000"	NULL
2	1	"2023-01-01 00:00:01"+0000"	"2023-01-01 00:00:00"+0000"
3	1	"2023-01-01 00:00:02"+0000"	"2023-01-01 00:00:01"+0000"
4	1	"2023-01-01 00:00:03"+0000"	"2023-01-01 00:00:02"+0000"
5	1	"2023-01-01 00:00:04"+0000"	"2023-01-01 00:00:03"+0000"
6	1	"2023-01-01 00:00:05"+0000"	"2023-01-01 00:00:04"+0000"
7	1	"2023-01-01 00:00:06"+0000"	"2023-01-01 00:00:05"+0000"
8	1	"2023-01-01 00:00:07"+0000"	"2023-01-01 00:00:06"+0000"
9	1	"2023-01-01 00:00:08"+0000"	"2023-01-01 00:00:07"+0000"
10	1	"2023-01-01 00:00:09"+0000"	"2023-01-01 00:00:08"+0000"
11	1	"2023-01-01 00:00:10"+0000"	"2023-01-01 00:00:09"+0000"
12	1	"2023-01-01 00:00:11"+0000"	"2023-01-01 00:00:10"+0000"
13	1	"2023-01-01 00:00:12"+0000"	"2023-01-01 00:00:11"+0000"
14	1	"2023-01-01 00:00:13"+0000"	"2023-01-01 00:00:12"+0000"
15	1	"2023-01-01 00:00:14"+0000"	"2023-01-01 00:00:13"+0000"
16	1	"2023-01-01 00:00:15"+0000"	"2023-01-01 00:00:14"+0000"
17	1	"2023-01-01 00:00:16"+0000"	"2023-01-01 00:00:15"+0000"
18	1	"2023-01-01 00:00:17"+0000"	"2023-01-01 00:00:16"+0000"
19	1	"2023-01-01 00:00:18"+0000"	"2023-01-01 00:00:17"+0000"
20	1	"2023-01-01 00:00:19"+0000"	"2023-01-01 00:00:18"+0000"

Now that we have the table looks like partition and order it, we can notice you will see that partitioning exists only inside each partition and it's not global.

name_id	user_id	t_timestamp	lag(t.timestamp)
1	1	"2023-01-01 00:00:00"+0000"	NULL
2	1	"2023-01-01 00:00:01"+0000"	"2023-01-01 00:00:00"+0000"
3	1	"2023-01-01 00:00:02"+0000"	"2023-01-01 00:00:01"+0000"
4	1	"2023-01-01 00:00:03"+0000"	"2023-01-01 00:00:02"+0000"
5	1	"2023-01-01 00:00:04"+0000"	"2023-01-01 00:00:03"+0000"
6	1	"2023-01-01 00:00:05"+0000"	"2023-01-01 00:00:04"+0000"
7	1	"2023-01-01 00:00:06"+0000"	"2023-01-01 00:00:05"+0000"
8	1	"2023-01-01 00:00:07"+0000"	"2023-01-01 00:00:06"+0000"
9	1	"2023-01-01 00:00:08"+0000"	"2023-01-01 00:00:07"+0000"
10	1	"2023-01-01 00:00:09"+0000"	"2023-01-01 00:00:08"+0000"
11	1	"2023-01-01 00:00:10"+0000"	"2023-01-01 00:00:09"+0000"
12	1	"2023-01-01 00:00:11"+0000"	"2023-01-01 00:00:10"+0000"
13	1	"2023-01-01 00:00:12"+0000"	"2023-01-01 00:00:11"+0000"
14	1	"2023-01-01 00:00:13"+0000"	"2023-01-01 00:00:12"+0000"
15	1	"2023-01-01 00:00:14"+0000"	"2023-01-01 00:00:13"+0000"
16	1	"2023-01-01 00:00:15"+0000"	"2023-01-01 00:00:14"+0000"
17	1	"2023-01-01 00:00:16"+0000"	"2023-01-01 00:00:15"+0000"
18	1	"2023-01-01 00:00:17"+0000"	"2023-01-01 00:00:16"+0000"
19	1	"2023-01-01 00:00:18"+0000"	"2023-01-01 00:00:17"+0000"
20	1	"2023-01-01 00:00:19"+0000"	"2023-01-01 00:00:18"+0000"

Now that we have the table looks like partition and order it, we can notice you will see that partitioning exists only inside each partition and it's not global.

What about Nulls? Do they matter?

Well nulls always matter! 🤓

We always should be aware of the null semantics around our window functions. Always check the documentation and see how the window function we care about is behaving in the presence of nulls.

For example in DuckDB some functions can be configured to ignore nulls although the default behavior for all window functions is to respect them. Such an example is the `LAG` function that we used earlier.

In any case, make sure you understand well the semantics of your functions and the data you are working with. What if an aggregation function meets a division by a null value?

Enough with theory, Let's have fun!

Ok, let's work on some examples using window functions. For these examples we will be using DuckDB.

First, download DuckDB if you haven't already. I'm assumed to have a Linux/Mac OS X machine but feel free to use any other OS we working with DuckDB.

We will be working with IMDB files so you will also need to install the IMDB extension for DuckDB. To do that, you just have to:

```
$duckdb
Enter ".help" for usage hints.
Connected to a transient in-memory database.
Use ".open FILENAME" to reopen on a persistent database.
D install imdb;
```

That's all, now you are ready to start playing around and being dangerous.

The case of Sessionization

We will use a very common problem that requires window functions: Session. We want to be pragmatic here, so using a real life example that you most probably will face sooner or later is what we are going for.

What is sessionization?

Assuming we have a number of user interactions captured in different moments in time, how can we group them into "meaningful" buckets. By meaningful in our context, we mean events that happened during one active session of the user.

🤓 **Typically a session is defined as:** all events that happened in less than 30 minutes from each other for a specific time.

The definition can get more complicated but this will suffice for our example. Sessions are very useful in many analytical scenarios. For example, Google Analytics is using it as the default session definition.

The data

Now that we have the tools and the problem, we just need data and we are ready to go.

Again, we will try to be as realistic as possible. We will be using customer event data captured in the Fivetran connector by Rudderstack and Segment.

These are the two most commonly used tools for capturing user interactions.

For more information on the whole scheme of this format, you should check the warehouse documentation provided here.

We are using data from this open-sourced dataset, provided, in case you'd like to generate your own data, you can use the tool :) ****** You can find it here.

I'll also include a sample file that you can use directly! Using the event generator is useful if you want to experiment with different number of users and events and work on performance of your queries.

The queries

The first thing we have to do is to load our data into DuckDB and see how they look like.

Here I'll assume that the file is named test.json and that it's in the same path that you run duckdb from. Feel free to play around with the name of the file and create a better group of the file if works and the SQL syntax of DuckDB.

```
D select * from read_ndjson('test.json') limit 1;
```

And if we execute the above, we'll see something like this as output:

```
┌──────────────
```

row_number() to get the time number and use it as the id for the event. See the use of the **over** keyword? That's an indication that row_number() is a window function.

In our case we didn't want to have partitions because that wouldn't generate globally unique ids, so we run the window function over the whole table.

Now that we have a way to generate unique ids let's figure out how to get rid of the **json** type and turn it into something more useful.

Consider the following SQL query:

```
D select cast(user_id as string) as user_id from extracted_json limit
1;

┌──────────────────────────────────────┐
│                user_id               │
│                varchar               │
├──────────────────────────────────────┤
│ "15474ff5-3e59-44fa-a075-13c1b2f9d181" │
└──────────────────────────────────────┘
```

The function **CAST** is what we need here. We ask DuckDB to take the JSON type and turn it into a String and in this case it worked perfectly as you can see by the result.

Now consider the following:

```
D select cast(original_timestamp as timestamp) as original_timestamp
from extracted_json
limit 1;

Error: Conversion Error: timestamp field value out of range:
"1970-01-19 15:00:17.100 UTC+",
expected format is (YYYY-MM-DD HH:MM:SS[.US][(+|-|A|H|M| ZONE)])
```

Ouch, we got an error! Apparently the format we used for the date cannot be converted into a timestamp. We need to fix this before we move on.

💡 These type of issues are extremely common when working with SQL and data in general, that's why I thought it would be good to actually have to figure this out as part of the exercise!

Check the following query:

```
SELECT
  CASE
  WHEN original_timestamp LIKE '%.%' THEN strptime(trim(both '"'
FROM original_timestamp), '%Y-%m-%d %H:%M:%S.%f UTC')
    ELSE strptime(trim(both '"' FROM original_timestamp), '%Y-%m-%d
%H:%M:%S UTC')
  END AS parsed_timestamp
FROM extracted_json limit 1;

┌──────────────────────┐
│   parsed_timestamp   │
│      timestamp       │
├──────────────────────┤
│ 1970-01-19 15:00:17.0001 │
└──────────────────────┘
```

we did it! So what happens with the above query.

1. First we need to trim the character " from both the beginning and the end of the value.
2. Then we need to account for two cases, one where milliseconds exist in time and one for when they don't. Again this is an issue with the generation of the data and we have to fix it here.
3. For each case, we use strptime to get a timestamp out of the string.

CASE is the equivalent of **IF-THEN-ELSE** statements in SQL.

Now that we have figured out everything, let's transform our raw data into something easier to work with by creating another view.

```
SELECT
  CASE
  WHEN original_timestamp LIKE '%.%' THEN strptime(trim(both '"' FROM
original_timestamp), '%Y-%m-%d %H:%M:%S.%f UTC')
    ELSE strptime(trim(both '"' FROM original_timestamp), '%Y-%m-%d
%H:%M:%S UTC')
    END AS p_timestamp,
    cast(user_id as string) as user_id,
    cast(event_type as string) as event_type,
    row_number() over () as event_id
  FROM extracted_json limit 1;
```

```
┌──────────────────────┬──────────────────────────────────────┬────────────┬────────────┐
│     p_timestamp      │               user_id                │ event_type │ event_id │
│      timestamp       │                varchar               │   varchar  │   int64    │
├──────────────────────┼──────────────────────────────────────┼────────────┼────────────┤
│ 1970-01-19 15:00:17.0001 │ "15474ff5-3e59-44fa-a075-13c1b2f9d181" │ "Page"     │          1 │
└──────────────────────┴──────────────────────────────────────┴────────────┴────────────┘
```

and we have what we need! Now let's create a view so we can work with it easily.

```
create view events as SELECT
  CASE
  WHEN original_timestamp LIKE '%.%' THEN strptime(trim(both '"' FROM
original_timestamp), '%Y-%m-%d %H:%M:%S.%f UTC')
    ELSE strptime(trim(both '"' FROM original_timestamp), '%Y-%m-%d
%H:%M:%S UTC')
    END AS p_timestamp,
    cast(user_id as string) as user_id,
    cast(event_type as string) as event_type,
    row_number() over () as event_id
  FROM extracted_json;
```

```
D select count(*) from events;

┌──────────┐
│ count_star() │
│    int64     │
├──────────┤
│    31415 │
└──────────┘
```

And

```
D describe events;
```

```
┌────────────┬────────────┬─────────┬─────────┬─────────┬─────────┐
│ column_name │ column_type │  null   │   key   │ default │ extra │
│   varchar  │   varchar  │ varchar │ int32 │  varchar  │ int32 │
├────────────┼────────────┼─────────┼─────────┼─────────┼─────────┤
│ p_timestamp │ TIMESTAMP  │ YES     │         │         │         │
│ user_id    │ VARCHAR    │ YES     │         │         │         │
│ event_type │ VARCHAR    │ YES     │         │         │         │
│ event_id   │ BIGINT     │ YES     │         │         │         │
└────────────┴────────────┴─────────┴─────────┴─────────┴─────────┘
```

We are good to go!

I know it's been a journey so far, but we already worked with a window function and we also did something important, cleaned and prepared our data!

This is big part of the work involved with data.

Now let's go back to sessions. Remember the definition of a session?

💡 A session is the set of events for a particular user that happened in less than 30 minutes between successive events.

If you remember the examples you gave earlier you might have already figured out that **LAG** is probably a great candidate for helping us with our problem here, let's see how.

consider the following query:

```
WITH events_enriched AS (
  SELECT
    user_id,
    p_timestamp,
    LAG(p_timestamp) OVER (PARTITION BY user_id ORDER BY p_timestamp
ASC) AS prev_timestamp
  FROM events
),
sessions AS (
  SELECT
    user_id,
    p_timestamp,
    prev_timestamp,
    SUM(CASE
      WHEN p_timestamp - prev_timestamp > interval '30 minutes' OR
prev_timestamp IS NULL THEN 1
      ELSE 0
    END) OVER (PARTITION BY user_id ORDER BY p_timestamp ASC) AS
session_id
  FROM events_enriched
)
SELECT user_id, p_timestamp, session_id
FROM sessions
ORDER BY user_id, p_timestamp ASC;
```

Here we are also using CTEs that we haven't talked yet, but don't worry that much if you find this **WITH** syntax new. It's mainly a way to organize the code and make it cleaner.

As you can see we start by enriching our events by adding the previous timestamp as a new column. To do that we of course use **LAG** and windows! The way that part of the query works should be clear to you by now.

The second part is the session creation. Here we are creating a new column that tracks the session id. The interesting part is what's inside the **SUM** clause.

Again here you see the beauty of the declarative nature of SQL. We can add a 0 or 1 based on the difference between the two columns that represent the event times.

Once again, we use **SUM** as a window function, remember that all aggregation functions are window functions, to calculate the session id for each user.

With that query we will end up with a result like this, I will limit the results to 10 for convenience.

```
┌──────────────────────────────────────┬────────────────────────┬────────────┐
│               user_id                │       p_timestamp      │ session_id │
│                varchar               │       timestamp        │   int128   │
├──────────────────────────────────────┼────────────────────────┼────────────┤
│ "0075658-dfaf-4921-883c-3387dac51608" │ 1970-01-19 00:22:03.00012 │          1 │
│ "0075658-dfaf-4921-883c-3387dac51608" │ 1970-01-19 00:22:03.000136 │          1 │
│ "0075658-dfaf-4921-883c-3387dac51608" │ 1970-01-19 00:01:56.00022 │          2 │
│ "0075658-dfaf-4921-883c-3387dac51608" │ 1970-01-19 00:31:11.00048 │          3 │
│ "0075658-dfaf-4921-883c-3387dac51608" │ 1970-01-19 09:31:26.0007  │          3 │
│ "0075658-dfaf-4921-883c-3387dac51608" │ 1970-01-19 10:10:40.00068 │          4 │
│ "0075658-dfaf-4921-883c-3387dac51608" │ 1970-01-19 10:27:29.00082 │          4 │
│ "0075658-dfaf-4921-883c-3387dac51608" │ 1970-01-19 10:55:42.0003  │          5 │
│ "0075658-dfaf-4921-883c-3387dac51608" │ 1970-01-19 13:27:45.00024 │          6 │
│ "0075658-dfaf-4921-883c-3387dac51608" │ 1970-01-19 13:27:45.000732 │          6 │
├──────────────────────────────────────┴────────────────────────┴────────────┤
│ 10 rows                                                          3 columns │
└────────────────────────────────────────────────────────────────────────────┘
```

Isn't pretty? 😁

Conclusion

Window functions are super powerful. If you master the concepts behind them you'll be able to write some very expressive and elegant SQL code.

They might require from you to change the way you are thinking, especially if you are coming from more imperative programming languages but it won't take that long to get comfortable with them.

I hope that the examples I gave were helpful!

In any case, please let me know of what you think and what else you'd like to see as a SQL tutorial.

References

DuckDB Intervals Documentation

DuckDB Timestamp Documentation

DuckDB Text Functions Documentation

DuckDB Date Formats Documentation

DuckDB JSON Extension Documentation

DuckDB Window Functions Documentation

DuckDB Installation Documentation

RudderStack Event Schema Spec

PostgreSQL Window Functions Documentation

Snowflake Window Functions Documentation

Fake Events Generator Source Code

Sample Event Data

If you liked this post, consider sharing it on Hacker News.

← Back to posts

© 2026 Kostas Pardalis